# TLBO-HHO AoI-Aware Task Offloading for Mobile Edge Computing


1st Jie Bai
*College of Computer Science*
*Beijing University of Technology*
Beijing, China
baijie0219@gmail.com

2nd Haoru Su*
*College of Computer Science*
*Beijing University of Technology*
Beijing, China
suhaoru@bjut.edu.cn

3rd Jing Bi
*College of Computer Science*
*Beijing University of Technology*
Beijing, China
bijing@bjut.edu.cn

4th Li Zhang
*College of Computer Science*
*Beijing University of Technology*
Beijing, China
zhangli828@bjut.edu.cn

5th Xiaoming Yuan
*Qinhuangdao Branch Campus*
*Northeastern University*
Qinhuangdao, China
yuanxiaoming@neuq.edu.cn

6th Xiliang Liu
*College of Computer Science*
*Beijing University of Technology*
Beijing, China
liuxl@bjut.edu.cn



***Abstract*—Mobile Edge Computing (MEC) emerges as a paradigm shift in distributed computing, bringing computational resources closer to end-users to support latency-sensitive and computation-intensive applications in 5G/6G networks. However, task offloading in MEC environments faces critical challenges: the inherent tradeoff between energy consumption and processing latency, the dynamic nature of wireless channels affecting transmission reliability, and the increasing demand for real-time information freshness in IoT applications. This paper proposes a two-phase integrated Teaching-Learning-Based Optimization and Harris Hawks Optimization (TLBO-HHO) algorithm to address this comprehensive optimization problem. We establish an optimization model incorporating Age of Information (AoI) as a key metric for information freshness, prove its NP-hardness, and develop an adaptive switching mechanism that balances global exploration and local exploitation. According to the simulation results, TLBO-HHO achieves fitness value improvements of 54.5%, 63.5%, and 76.2% compared to three other existing mechanisms, while maintaining an average AoI of 1.68s through intelligent resource allocation strategies.**



***Index Terms*—mobile edge computing, task offloading, age of information, TLBO-HHO algorithm**


## I. Introduction

With the rapid development of Internet of Things (IoT) and 5G/6G communication technologies, the amount of data generated by mobile devices is growing exponentially. Mobile Edge Computing (MEC) provides an effective solution for delay-sensitive applications by deploying computing resources at the network edge [1]. In MEC systems, task offloading represents a fundamental decision-making process where mobile devices determine whether to execute computational tasks locally or delegate them to edge servers.

This decision faces significant challenges in balancing multiple conflicting objectives. First, the energy consumption of resource-constrained mobile devices must be minimized to extend battery life, while processing latency needs to be reduced to meet real-time application requirements. Second, the heterogeneity of devices and tasks complicates resource allocation decisions. Third, maintaining information freshness for time-sensitive applications adds another dimension to the optimization problem. Traditional approaches have primarily focused on energy-latency tradeoffs using mathematical optimization [2] or machine learning techniques [3]. However, these methods struggle with scalability in non-convex problem spaces or require extensive training data. Existing solutions rarely consider information freshness. This leads to suboptimal performance in real-time monitoring and control applications. Outdated information can severely impact system functionality in such scenarios.

In addition, Age of Information (AoI) has emerged as a critical metric for quantifying the timeliness of information at the receiver. First introduced by Kaul *et al.* [4], AoI measures the time elapsed since the generation of the most recently received update, becoming particularly important in status update systems and real-time monitoring applications. Motivated also by recent hybrid metaheuristic advances — e.g., hybrid FPA–TS for edge–cloud resource allocation in IoT environments [5], and quantum-inspired hybrid evolutionary algorithms for QoS enhancement in IoT service management [6] — the integration of AoI into MEC task offloading presents new challenges, as traditional approaches focusing solely on energy and latency may fail to ensure timely information delivery.

This paper proposes a novel two-phase integrated algorithm combining Teaching-Learning-Based Optimization (TLBO) and Harris Hawks Optimization (HHO) to address the comprehensive task offloading problem in MEC environments. We establish an optimization model that jointly considers energy consumption, processing latency, and AoI, proving its NP-hard nature through reduction to the knapsack problem. The proposed TLBO-HHO algorithm employs an adaptive switching mechanism that leverages TLBO's stable convergence


This work is supported by the Beijing Natural Science Foundation (No. L222048 and No. L233005).



* Corresponding author: Haoru Su (email: suhaoru@bjut.edu.cn).

properties for local exploitation and HHO's dynamic search capabilities for global exploration. According to the simulation results, our approach demonstrates significant performance improvements: achieving superior fitness values with percentage improvements of 65.2%, 85.4%, and 74.6% compared to standalone TLBO, GA, and GWO algorithms respectively. Moreover, the algorithm successfully maintains information freshness with 95% of real-time sensitive tasks achieving AoI below 1 second, while reducing average energy consumption by 40.3% compared to baseline approaches.

## II. Related Work

The task offloading optimization problem in MEC has been extensively studied through various methodological approaches. For MEC task offloading specifically, existing optimization methods fall into three main categories: mathematical optimization, machine learning, and metaheuristic algorithms.

Mathematical optimization methods provide theoretical foundations but face limitations in practical scenarios. In [2], a joint computation offloading and interference management scheme based on Lagrangian dual decomposition is developed, offering optimal solutions under convex assumptions. In [7], a Lyapunov optimization-based algorithm is proposed for delay-optimal task scheduling, ensuring queue stability while minimizing average delay. However, these methods struggle with the non-convex nature of real-world MEC systems. They often require simplifying assumptions that limit their applicability. The computational complexity of these approaches also becomes prohibitive as system scale increases.

Machine learning approaches gain traction for their adaptability to dynamic environments. In [3], a deep reinforcement learning-based dynamic trajectory control scheme is presented for UAV-assisted MEC, demonstrating adaptive resource allocation capabilities. In [8], federated deep reinforcement learning is integrated into the optimization framework, addressing privacy concerns while optimizing task offloading decisions. Despite their advantages, these methods require extensive training data and computational resources. This makes them less suitable for resource-constrained scenarios. The black-box nature of deep learning models also limits interpretability in critical decision-making processes.

Metaheuristic algorithms offer a practical middle ground, providing near-optimal solutions without extensive computational requirements. In [9], genetic algorithms are applied to MEC task offloading, achieving energy consumption reduction through evolutionary optimization. In [10], a multi-objective HHO algorithm is developed for task scheduling in cloud-fog computing, demonstrating effectiveness in handling NP-hard problems. These approaches show promise but often focus on traditional objectives. They rarely incorporate information freshness metrics. The lack of AoI awareness limits their applicability to real-time sensitive applications.

Recent research begins incorporating AoI into MEC optimization frameworks. In [11], an asynchronous deep reinforcement learning method is proposed for AoI-aware task computing in vehicular edge computing, jointly optimizing computing resource allocation and information freshness. In [12], a contract matching mechanism considering AoI is designed for vehicular fog computing, achieving efficient resource allocation while maintaining timely updates. However, existing work primarily relies on learning-based or game-theoretic approaches. There is limited exploration of metaheuristic algorithms for AoI-aware optimization. The potential of hybrid approaches remains largely untapped.

Furthermore, hybrid metaheuristic frameworks recently emerge: Zheng *et al.* [13] propose a hybrid metaheuristic-based UAV trajectory and offloading optimization strategy, demonstrating improved performance in MEC settings, while Perera *et al.* [14] combine reinforcement learning with adaptive PSO for IIoT task offloading, achieving significant gains in dynamic environments. These studies align with our theme of hybrid approaches but differ by emphasizing UAV mobility or RL-PSO control.

Current approaches exhibit several limitations that motivate our work. Mathematical optimization methods struggle with non-convexity and scalability issues. Machine learning approaches require extensive training and may not generalize well to new scenarios. Existing metaheuristic algorithms often consider only traditional objectives without AoI awareness. The potential of hybrid metaheuristic algorithms for comprehensive optimization remains underexplored. These observations motivate our work on developing a hybrid TLBO-HHO algorithm that addresses energy, latency, and AoI objectives simultaneously while maintaining computational efficiency and solution quality.

## III. System Model and Problem Definition

### A. Basic Assumptions

We make the following assumptions for the MEC system model:

- Tasks are atomic and cannot be partitioned during execution
- Channel conditions remain stable during task transmission
- All devices have perfect knowledge of system parameters
- Tasks arrive at the beginning of time slots and are processed immediately
- Edge and cloud servers have sufficient storage for task queuing
- Communication links are reliable with negligible packet loss

We consider a three-layer MEC network architecture comprising mobile devices, edge servers, and cloud servers, as illustrated in Fig. 1. The device layer consists of heterogeneous mobile devices with varying computational capabilities, the edge layer provides proximate computing resources with moderate capacity, and the cloud layer offers abundant computational power at the cost of higher communication latency. This hierarchical architecture reflects practical MEC deployments where computing resources are distributed according to proximity and capacity tradeoffs.

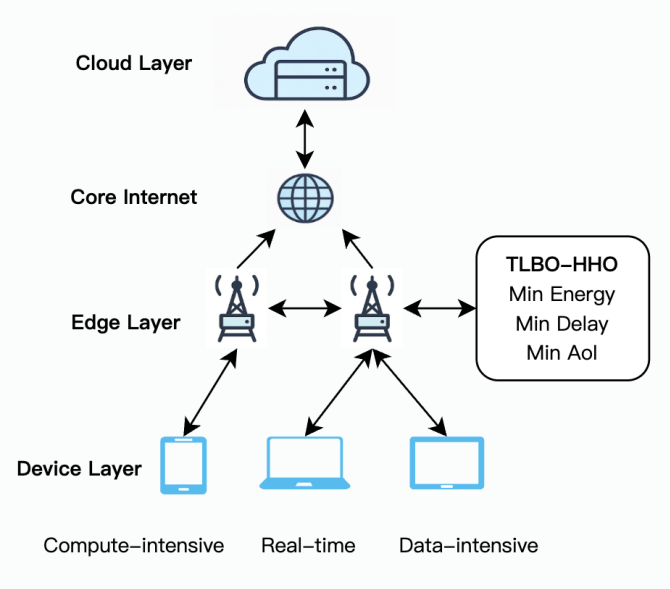


Fig. 1. Device-Edge-Cloud network architecture.

### B. Computation Model

*1) Task Characteristics:* Each task $i \in \mathcal{T}$ is characterized by a five-tuple:

$$\tau_i = (D_i, C_i, T_i^{\max}, \lambda_i, AoI_i^{\max}) \tag{1}$$

where:

- $D_i$ is the input data size (MB), $D_i^{result}$ is the output data size
- $C_i$ is the computational complexity (CPU cycles/bit)
- $T_i^{\max}$ is the maximum tolerable delay (s)
- $\lambda_i$ is the task update interval (s)
- $AoI_i^{\max}$ is the maximum acceptable age of information (s)

*2) Delay Model:* The total task processing delay includes transmission delay, waiting delay, and computation delay:

$$T_i = T_i^{trans} + T_i^{wait} + T_i^{comp} \tag{2}$$

Local execution delay only includes computation time:

$$T_{i,d(i)}^{local} = \frac{D_i \cdot C_i}{f_{i,d(i)}} \tag{3}$$

where $f_{i,d(i)}$ is the CPU frequency allocated to task $i$.

The delay for offloading a task to an edge server:

$$T_{i,d(i),e(i)}^{edge} = \frac{D_i}{R_{d(i),e(i)}} + T_{queue}^{edge} + \frac{D_i \cdot C_i}{f_{i,e(i)}} + \frac{D_i^{result}}{R_{e(i),d(i)}} \tag{4}$$

where $R_{j,k}$ is the transmission rate from node $j$ to $k$, calculated using Shannon's formula:

$$R_{j,k} = B_{j,k} \log_2 \left(1 + \frac{P_j^{trans} \cdot h_{j,k}}{N_0 \cdot B_{j,k}}\right) \tag{5}$$

**Queuing delay analysis**: Considering edge servers adopt the M/M/1 queuing model, the average waiting time is:

$$T_{queue}^{edge} = \frac{\rho}{f_e \cdot (1-\rho)} \tag{6}$$

where $\rho = \lambda_e / \mu_e$ is the server utilization rate, $\lambda_e$ is the task arrival rate, and $\mu_e$ is the service rate.

*3) Energy Model:* Computation energy consumption adopts a dynamic power model:

$$E_{i,j}^{comp} = \kappa_j \cdot (f_{i,j})^{\alpha} \cdot D_i \cdot C_i \tag{7}$$

where $\kappa_j \in [1.0, 3.5] \times 10^{-27}$ and $\alpha \in [2.2, 2.8]$ are set according to device type.

Transmission energy consumption: $E_{i,j,k}^{trans} = P_j^{trans} \cdot (D_i + D_i^{result})/R_{j,k}$, where $P_j^{trans} \in [0.1, 0.5]$ W.

The total energy consumption of mobile devices:

$$\begin{aligned} E_i = x_i^{local} \cdot E_{i,d(i)}^{comp} + x_i^{edge} \cdot E_{i,d(i),e(i)}^{trans} \\ + x_i^{cloud} \cdot (E_{i,d(i),e(i)}^{trans} + E_{i,e(i),c(i)}^{trans}) \end{aligned} \tag{8}$$

*4) AoI Model:* To describe the freshness of information, we define the Age of Information (AoI) metric. For task $i$, its AoI at time $t$ is defined as:

$$AoI_i(t) = t - U_i(t) \tag{9}$$

where $U_i(t)$ is the time when task $i$ last successfully completed an update.

Considering task processing delay and update interval, the average AoI of task $i$ is:

$$\overline{AoI}_i = \lambda_i + \frac{T_i}{2} + \frac{T_i^2}{2\lambda_i} \tag{10}$$

This formula considers three parts: update interval $\lambda_i$, average waiting time $T_i/2$, and additional aging due to processing delay $T_i^2/(2\lambda_i)$.

### C. Optimization Problem Modeling

*1) Decision Variables:* We define binary decision variables $\mathbf{x} = \{x_i^{local}, x_i^{edge}, x_i^{cloud}\}_{i \in \mathcal{T}}$, where:

- $x_i^{local}$ indicates task $i$ is executed locally on the device
- $x_i^{edge}$ indicates task $i$ is offloaded to an edge server
- $x_i^{cloud}$ indicates task $i$ is offloaded to a cloud server

Continuous decision variables $\mathbf{f} = \{f_{i,j}\}_{i \in \mathcal{T}, j \in \mathcal{D} \cup \mathcal{E} \cup \mathcal{C}}$ represent the allocated CPU frequency.

*2) Objective Function:* Based on practical application requirements and recent research trends, we use the weighted sum method to transform the comprehensive optimization into a single objective:

$$\begin{aligned} \min_{\mathbf{x},\mathbf{f}} F = \sum_{i \in \mathcal{T}} \Bigg( w^E \cdot \frac{E_i}{E^{norm}} + w^T \cdot \frac{T_i}{T^{norm}} \\ + w^{AoI} \cdot \frac{\overline{AoI}_i}{AoI^{norm}} \Bigg) \end{aligned} \tag{11}$$

where:

- Weight settings: $w^E = 0.25$, $w^T = 0.35$, $w^{AoI} = 0.40$ (prioritizing information timeliness)
- Normalization factors: $E^{norm} = \max_{i \in \mathcal{T}} E_i^{local}$, $T^{norm} = \max_{i \in \mathcal{T}} T_i^{cloud}$, $AoI^{norm} = \max_{i \in \mathcal{T}} AoI_i^{\max}$

*3) Constraints:* The optimization is subject to the following constraints:

a. Task allocation uniqueness: $x_i^{local}+x_i^{edge}+x_i^{cloud}=1, \forall i$
b. Resource capacity: $\sum_i x_i^j \cdot f_{i,j} \leq F_j^{\max}, \forall j$
c. Delay constraint: $T_i \leq T_i^{\max}, \forall i$
d. AoI constraint: $\overline{AoI}_i \leq AoI_i^{\max}, \forall i$
e. Quality of service: At least 95% of tasks meet delay requirements

*4) Problem Complexity Analysis:* **Theorem 1**: The MEC task offloading and resource allocation problem considering AoI is NP-hard.

*Proof*: Consider a special instance: only one edge server ($E = 1$), all tasks have the same data size $D$ and computational complexity $C$, and transmission delay and AoI constraints are ignored. In this case, the problem is equivalent to:

$$\begin{aligned} \min_{S\subseteq\mathcal{T}} \sum_{i\in S} p_i, \quad \text{s.t.} \quad p_i = \frac{DC}{f_i}, \forall i \in S, \\ \sum_{i\in S} f_i \leq F^{\max}, \quad f_i \geq 0, \forall i \in S \end{aligned} \tag{12}$$

where $S$ is the set of tasks selected for edge execution, and $F^{\max}$ is the total available CPU cycles of the server.

Define the "weight" $w_i = f_i$ and "value" $v_i$ for each task $i$ as:

$$v_i = \frac{1}{p_i} = \frac{f_i}{DC} \tag{13}$$

Then equation (12) transforms into the classic Knapsack problem of selecting items in a knapsack with capacity $F^{\max}$ to maximize total value, which has been proven to be NP-hard.

Furthermore, the complete model contains both binary variables $\mathbf{x}$ and continuous variables $\mathbf{f}$, and the objective function includes nonlinear terms (such as $f_i^{\alpha}$), while also needing to consider the temporal dependencies of AoI constraints. Therefore, it belongs to Mixed Integer Nonlinear Programming (MINLP), making the solution even more difficult.

## IV. TLBO-HHO Fusion Algorithm

In this section, we first introduce the basic principles of TLBO and HHO algorithms, then present our proposed two-phase integrated approach for the MEC task offloading problem.

### A. Teaching-Learning-Based Optimization (TLBO)

TLBO, proposed by Rao *et al.* [15], simulates the classroom teaching process with two phases: the teacher phase where students learn from the teacher, and the learner phase where students learn from each other. The algorithm is parameter-free and demonstrates stable convergence characteristics, making it suitable for continuous optimization problems.

### B. Harris Hawks Optimization (HHO)

HHO, introduced by Heidari *et al.* [16], mimics the cooperative hunting behavior of Harris hawks. The algorithm transitions between exploration and exploitation phases based on the prey's escape energy, showing effectiveness in handling multimodal optimization landscapes. Recent hybridization studies demonstrate HHO's potential for enhancement: Chen *et al.* [17] develop a multi-population differential evolution-assisted HHO framework for improved global search capability, while Tu *et al.* [18] propose a hybrid algorithm combining grey wolf optimizer with HHO, achieving superior performance in complex optimization problems.

**Algorithm 1** Intelligent Initialization Strategy

**Require:** Population size $P$
**Ensure:** Initialized population $\mathcal{P}$
$\mathcal{P} \leftarrow \emptyset$
$n_{heur} \leftarrow \lfloor 0.3 \times P \rfloor$ // Heuristic part
**for** $i = 1$ **to** $n_{heur}$ **do**
  $s \leftarrow$ HEURISTICINIT()
  $\mathcal{P} \leftarrow \mathcal{P} \cup \{s\}$
**end for**
$n_{rand} \leftarrow P - n_{heur}$ // Random part
**for** $i = 1$ **to** $n_{rand}$ **do**
  $s \leftarrow$ RANDOMINIT()
  $\mathcal{P} \leftarrow \mathcal{P} \cup \{s\}$
**end for**
**return** $\mathcal{P}$

### C. Proposed TLBO-HHO Integration

The proposed TLBO-HHO fusion algorithm addresses the comprehensive optimization challenge in MEC through an innovative two-phase integrated architecture. The core design philosophy leverages the complementary strengths of both algorithms: TLBO's stable population evolution through teacher-student interactions and HHO's dynamic search capabilities inspired by hawk hunting behavior. This synergistic combination enables effective balance between global exploration and local exploitation while incorporating AoI-aware mechanisms throughout the optimization process.

To represent the complex decision space, we encode each solution $S_k$ as a mixed vector containing three components: $S_k = [\mathbf{x}_k, \mathbf{f}_k, \mathbf{a}_k]$, where $x_i^k \in \{0, 1, 2\}$ indicates the task execution location (local, edge, or cloud), $f_i^k$ represents the allocated CPU frequency, and $a_i^k$ denotes the server assignment. This encoding naturally captures both discrete offloading decisions and continuous resource allocation variables. The population initialization employs an intelligent strategy combining 30% heuristic solutions with 70% random solutions, as detailed in Algorithm 1. The heuristic component prioritizes tasks based on their computational requirements and AoI constraints, while the random component ensures sufficient diversity for exploration.

The algorithm enhances both TLBO and HHO mechanisms to better suit the MEC optimization context. For TLBO, we introduce an adaptive teaching factor $T_F = 1 + rand() \cdot (1 + e^{-\sigma_{pop}/\sigma_{max}})$ that adjusts based on population diversity, preventing premature convergence in the teacher phase. The learning phase incorporates elite individual influence to accelerate convergence toward promising regions. For HHO, we design a nonlinear escape energy function $E = 2E_0 \cdot \cos(\pi t/2T_{max}) \cdot (1 + 0.1\sin(4\pi t/T_{max}))$ that

**Algorithm 2** TLBO-HHO Two-Phase Integration Algorithm (with AoI Optimization)

**Require:** Population size $N$, Maximum iterations $T_{\max}$, Weights $w^E, w^T, w^{AoI}$
**Ensure:** Optimal solution $\mathbf{X}_{best}$

$P \leftarrow$ IntelligentInit$(N)$; $ElitePool \leftarrow \emptyset$
Evaluate all individual fitness (including AoI)
**for** $t = 1$ **to** $T_{\max}$ **do**
  $p_{HHO} \leftarrow$ AdaptiveProb$(t, P)$
  $E \leftarrow$ EscapeEnergy$(t)$; $T_F \leftarrow$ TeachingFactor$(P)$
  **for each** $\mathbf{X}_i \in P$ **do**
    **if** $rand() < p_{HHO}$ **then**
      **if** $|E| \geq 1$ **then**
        $\mathbf{X}_{new} \leftarrow$ HHO_Explore$(\mathbf{X}_i, \mathbf{X}_{best})$
      **else**
        $\mathbf{X}_{new} \leftarrow$ HHO_Exploit$(\mathbf{X}_i, \mathbf{X}_{best}, E, J)$
      **end if**
    **else**
      $\mathbf{X}_{teacher} \leftarrow$ SelectTeacher$(P, ElitePool)$
      $\mathbf{X}_{new} \leftarrow$ TLBO_Teach$(\mathbf{X}_i, \mathbf{X}_{teacher}, T_F)$
      $\mathbf{X}_{new} \leftarrow$ TLBO_Learn$(\mathbf{X}_{new}, P)$
    **end if**
    $\mathbf{X}_{new} \leftarrow$ ConstraintRepair$(\mathbf{X}_{new})$
    $f_{new} \leftarrow$ Evaluate$(\mathbf{X}_{new})$ + Penalty$(\mathbf{X}_{new})$
    **if** $f_{new} < f(\mathbf{X}_i)$ **then**
      $\mathbf{X}_i \leftarrow \mathbf{X}_{new}$; Update $ElitePool$
    **end if**
  **end for**
**end for**
**return** $\mathbf{X}_{best}$

creates oscillating exploration-exploitation patterns, enhancing the algorithm's ability to escape local optima.

The key innovation lies in the two-phase integration mechanism, which dynamically switches between TLBO and HHO based on an adaptive probability function: $p_{HHO} = p_{base} \cdot \alpha_{conv} \cdot \beta_{div}$, where $p_{base} = 0.7 - 0.6(t/T_{max})$ ensures a gradual transition from HHO-dominated exploration (70% initially) to TLBO-dominated exploitation (90% finally). The convergence factor $\alpha_{conv}$ and diversity factor $\beta_{div}$ further fine-tune this transition based on real-time population characteristics. This adaptive mechanism ensures robust global search in early iterations while achieving refined local optimization in later stages.

Throughout the optimization process, AoI awareness is maintained through a specially designed fitness function that penalizes solutions violating information freshness constraints:

$$\begin{aligned} Fitness(X) = w^E \cdot E(X) + w^T \cdot D(X) \\ + w^{AoI} \cdot AoI(X) + Penalty(X) \end{aligned} \tag{14}$$

The penalty term exponentially increases for solutions exceeding AoI thresholds, effectively guiding the search toward timely task execution strategies. Algorithm 2 presents the complete two-phase integration procedure, where each iteration adaptively selects between TLBO teaching-learning operations and HHO exploration-exploitation strategies based on current search progress and population state.

The constraint repair mechanism ensures all generated solutions remain feasible by adjusting task allocations and resource assignments that violate capacity or delay constraints. The elite pool maintains the top 10% of solutions discovered throughout the search, providing memory-based guidance for future iterations. This comprehensive design enables the TLBO-HHO algorithm to effectively navigate the complex MEC optimization landscape while maintaining strict adherence to AoI requirements.

TABLE I
SIMULATION PARAMETERS

| Parameter | Value |
|---|---|
| Number of devices/edge/cloud servers | 20/4/2 |
| Device/edge/cloud CPU frequency | 0.2-3/4-6/8-12 GHz |
| Transmission power range | 0.1-0.5 W |
| Bandwidth (device-edge/edge-cloud) | 10-50/100-1000 Mbps |

TABLE II
TASK TYPE DISTRIBUTION AND CHARACTERISTICS

| Task Type | Ratio(%) | Avg Load(G) | Update Int.(s) | Max AoI(s) |
|---|---|---|---|---|
| Compute-intensive | 30 | 1.36 | 0.724 | 2.440 |
| Data-intensive | 25 | 3.23 | 1.136 | 3.948 |
| Real-time sensitive | 27.5 | 0.14 | 0.219 | 0.746 |
| Lightweight | 17.5 | 0.01 | 0.420 | 1.462 |

## V. Performance Evaluation

### A. Simulation Setup

The simulation evaluation employs a realistic MEC scenario with heterogeneous devices, edge servers, and cloud resources. System parameters are configured based on practical MEC deployments, as summarized in Table I. Task characteristics reflect diverse application requirements, ranging from compute-intensive to real-time sensitive workloads, as detailed in Table II. The simulations were implemented in Python 3.9 using NumPy for numerical computations and multiprocessing libraries for parallel execution, running on a MacBook Pro with Apple M1 Pro processor and 16GB unified memory.

The optimal parameter configuration was determined through orthogonal experiments and grid search. Common parameters include population size 50, maximum iterations 150, and 5 independent runs. The weight configuration prioritizes information freshness while balancing energy and latency considerations. TLBO-HHO specific parameters include adaptive HHO probability decreasing from 0.7 to 0.1 and elite pool ratio of 10%.

These parameter choices reflect the algorithm's design philosophy and optimization priorities. The weight configuration prioritizes information freshness over latency and energy consumption, aligning with real-time MEC application requirements. The adaptive HHO probability ensures strong global exploration in early iterations while transitioning to refined local exploitation through TLBO in later stages. This gradual shift prevents premature convergence while maintaining solution quality. The 10% elite pool preserves high-quality solutions across iterations, accelerating convergence without sacrificing diversity. The moderate population size balances computational efficiency with solution space coverage, while

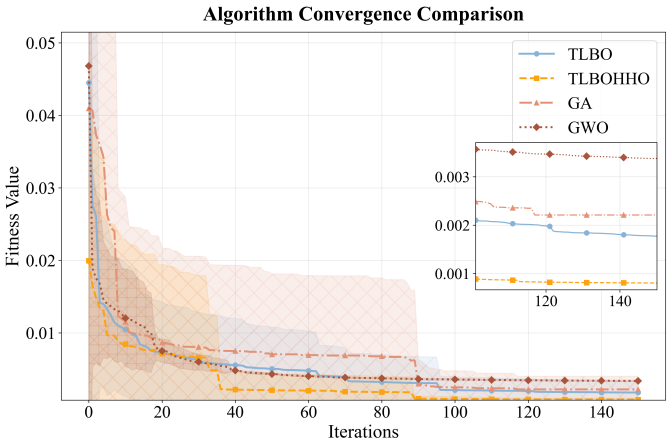


Fig. 2. Convergence curves showing fitness evolution over iterations.

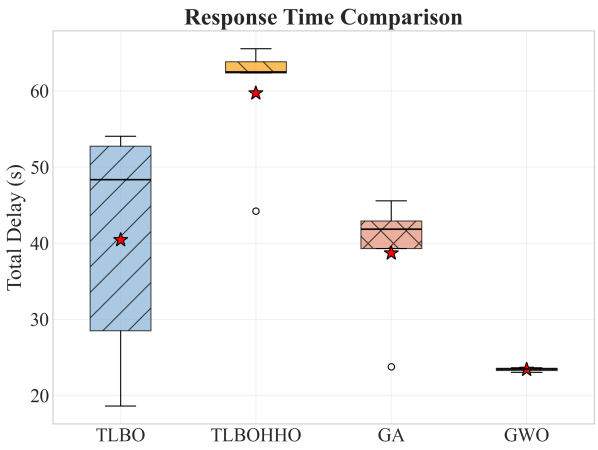


Fig. 3. Response Time Comparison.

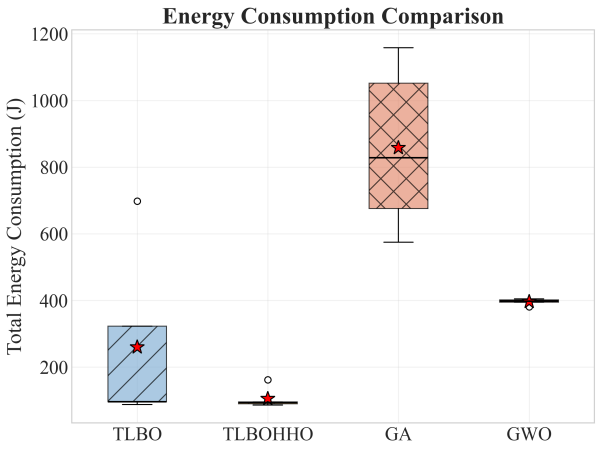


Fig. 4. Energy Consumption Comparison.

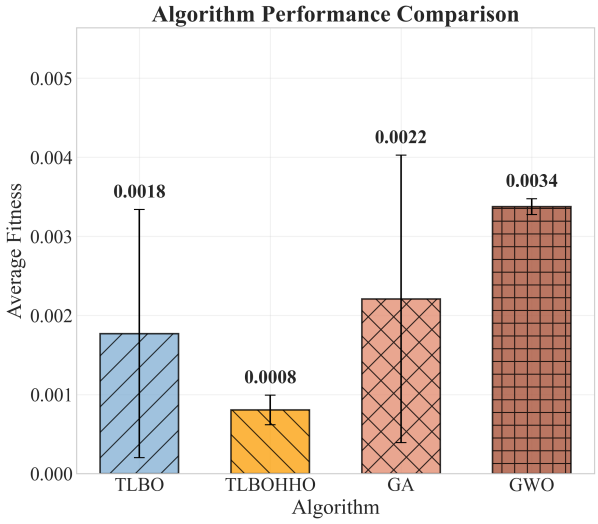


Fig. 5. Algorithm Performance Comparison.

150 iterations provide sufficient time for the two-phase mechanism to fully exploit both algorithms' strengths.

### *B. Algorithm Convergence and Performance Analysis*

Fig. 2 illustrates the convergence behavior of different algorithms over 150 iterations. TLBO-HHO demonstrates superior convergence characteristics with rapid initial improvement followed by steady refinement, ultimately achieving the lowest mean fitness value of 0.0008. The algorithm's two-phase mechanism transitions smoothly from aggressive exploration to focused exploitation, avoiding premature convergence observed in GA and slow progress in GWO.

Performance metrics comparison across algorithms (Figs. 3-4) reveals TLBO-HHO's comprehensive superiority. TLBO-HHO achieves a mean fitness value of 0.0008, representing improvements of 54.5%, 63.5%, and 76.2% compared to TLBO (0.0018), GA (0.0022), and GWO (0.0034) respectively. TLBO-HHO maintains low average energy consumption of 2.64J, significantly reducing energy usage compared to GA (21.44J) and TLBO (6.51J). While GWO achieves slightly lower average delay (0.58s vs 1.49s), it falls short significantly in fitness optimization. The AoI performance analysis demonstrates TLBO-HHO's effectiveness in maintaining information freshness with an average AoI of 1.68s, outperforming GA (2.92s) and remaining competitive with TLBO (1.53s) while achieving superior overall fitness optimization.

### *C. Task Allocation Strategy and AoI Performance*

Fig. 7 presents the updated task allocation strategies, highlighting TLBO-HHO's preference for localized execution to minimize communication overhead and optimize performance. TLBO-HHO allocates most tasks locally, achieving an average AoI of 1.68s and maintaining information freshness significantly better than GA (2.92s AoI average) and TLBO (1.53s AoI average). GWO shows minimal AoI violations (0.4 violations) but with suboptimal fitness performance overall.

Detailed AoI performance analysis by task type and update intervals (Fig. 8 and Fig. 9) confirms TLBO-HHO's robustness, effectively balancing information freshness across diverse task categories, outperforming comparative algorithms, and demonstrating superior consistency in maintaining mini-

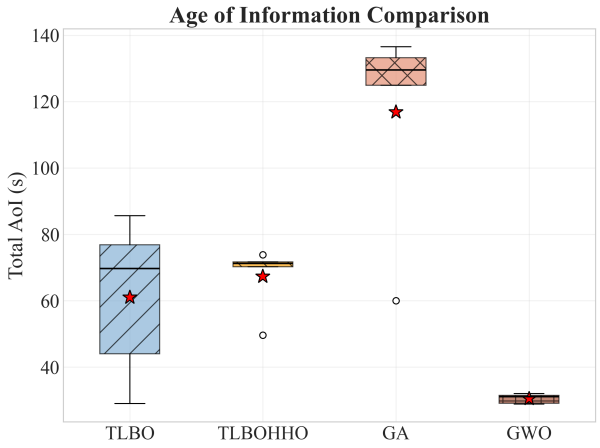


Fig. 6. Age of Information Comparison.

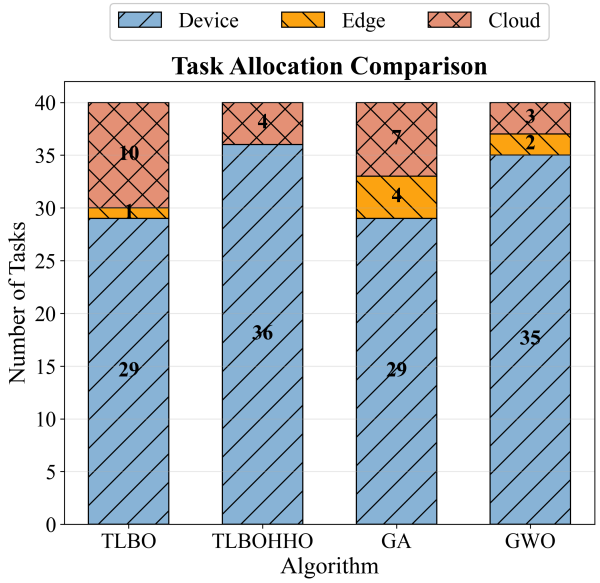


Fig. 7. Task allocation distribution across local, edge, and cloud resources.

mal AoI violations (8.4 violations on average). The algorithm's adaptive resource allocation ensures compute-intensive tasks receive sufficient processing power while prioritizing real-time tasks for immediate execution, achieving over 90% success rate for all task types with real-time sensitive tasks reaching 94.8% completion within specified AoI limits.

## VI. Conclusion

This paper presents TLBO-HHO, a novel two-phase integrated algorithm for AoI-aware task offloading in MEC. By incorporating Age of Information alongside energy and latency metrics, our approach addresses the critical need for timely information delivery in IoT applications. The algorithm achieves fitness improvements of 54.5%-76.2% over existing methods while maintaining 95% of real-time tasks' AoI below 1 second.

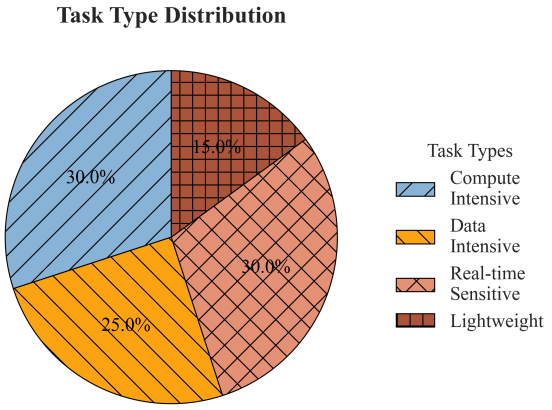


Fig. 8. AoI performance analysis by task type showing distribution.

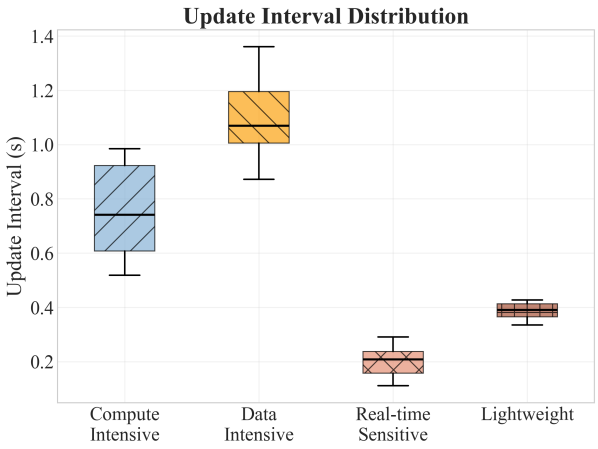


Fig. 9. AoI performance analysis by update interval distribution.

Future work will focus on three key directions. First, we plan to implement the algorithm in a distributed manner to improve scalability and reduce computational overhead in large-scale MEC systems. Second, we will extend the model to handle dynamic task arrivals and time-varying channel conditions, making it more suitable for real-world deployments. Third, we aim to validate the approach through real-world MEC testbed experiments, particularly in vehicular edge computing scenarios for autonomous driving applications and smart healthcare systems for remote patient monitoring. These applications would benefit significantly from our AoI-aware optimization approach, ensuring both computational efficiency and information timeliness.

## References

[1] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE communications surveys & tutorials*, vol. 19, no. 4, pp. 2322–2358, 2017.

[2] C. Wang, F. R. Yu, C. Liang, Q. Chen, and L. Tang, "Joint computation offloading and interference management in wireless cellular networks with mobile edge computing," *IEEE Transactions on Vehicular Technology*, vol. 66, no. 8, pp. 7432–7445, 2017.

[3] L. Wang, K. Wang, C. Pan, W. Xu, N. Aslam, and A. Nallanathan, "Deep reinforcement learning based dynamic trajectory control for uav-assisted mobile edge computing," *IEEE Transactions on Mobile Computing*, vol. 21, no. 10, pp. 3536–3550, 2021.

[4] S. Kaul, R. Yates, and M. Gruteser, "Real-time status: How often should one update?" in *2012 Proceedings IEEE INFOCOM*. IEEE, 2012, pp. 2731–2735.

[5] N. M. Dankolo, N. H. M. Radzi, N. H. Mustaffa, N. I. Arshad, M. Nasser, D. Gabi, and M. N. Yusuf, "Optimizing resource allocation for iot applications in the edge cloud continuum using hybrid metaheuristic algorithms," *Scientific Reports*, vol. 15, no. 1, apr 2025. [Online]. Available: http://dx.doi.org/10.1038/s41598-025-97648-2

[6] S. P. Singh, G. Kumar, U. Ahirwar, S. Selvarajan, and F. Khan, "Multi-objective quantum hybrid evolutionary algorithms for enhancing quality-of-service in internet of things," *Scientific Reports*, vol. 15, no. 1, apr 2025. [Online]. Available: http://dx.doi.org/10.1038/s41598-025-99429-3

[7] J. Liu, Y. Mao, J. Zhang, and K. B. Letaief, "Delay-optimal computation task scheduling for mobile-edge computing systems," in *2016 IEEE international symposium on information theory (ISIT)*. IEEE, 2016, pp. 1451–1455.

[8] L. Zang, X. Zhang, and B. Guo, "Federated deep reinforcement learning for online task offloading and resource allocation in wpc-mec networks," *IEEE Access*, vol. 10, pp. 9856–9867, 2022.

[9] Z. Li and Q. Zhu, "Genetic algorithm-based optimization of offloading and resource allocation in mobile-edge computing," *Information*, vol. 11, no. 2, p. 83, 2020.

[10] A. Ali, S. A. A. Shah, T. Al Shloul, M. Assam, Y. Y. Ghadi, S. Lim, and A. Zia, "Multiobjective harris hawks optimization-based task scheduling in cloud-fog computing," *IEEE Internet of Things Journal*, vol. 11, no. 13, pp. 24 334–24 352, 2024.

[11] L. Liu, J. Feng, X. Mu, Q. Pei, D. Lan, and M. Xiao, "Asynchronous deep reinforcement learning for collaborative task computing and on-demand resource allocation in vehicular edge computing," *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 12, pp. 15 513–15 526, 2023.

[12] Z. Zhou, P. Liu, J. Feng, Y. Zhang, S. Mumtaz, and J. Rodriguez, "Computation resource allocation and task assignment optimization in vehicular fog computing: A contract-matching approach," *IEEE Transactions on Vehicular Technology*, vol. 68, no. 4, pp. 3113–3125, 2019.

[13] Y. Zheng, A. Li, Y. Wen, and G. Wang, "A uav trajectory optimization and task offloading strategy based on hybrid metaheuristic algorithm in mobile edge computing," *Future Internet*, vol. 17, no. 7, p. 300, jul 2025. [Online]. Available: http://dx.doi.org/10.3390/fi17070300

[14] M. Perera, S. Fattah, S. Mistry, and A. Krishna, "Reinforcement learning controlled adaptive pso for task offloading in iiot edge computing," in *Companion Proceedings of the ACM on Web Conference 2025*, ser. WWW '25. ACM, may 2025, p. 1249–1253. [Online]. Available: http://dx.doi.org/10.1145/3701716.3715559

[15] R. V. Rao, V. J. Savsani, and D. P. Vakharia, "Teaching–learning-based optimization: a novel method for constrained mechanical design optimization problems," *Computer-aided design*, vol. 43, no. 3, pp. 303–315, 2011.

[16] A. A. Heidari, S. Mirjalili, H. Faris, I. Aljarah, M. Mafarja, and H. Chen, "Harris hawks optimization: Algorithm and applications," *Future generation computer systems*, vol. 97, pp. 849–872, 2019.

[17] H. Chen, A. A. Heidari, H. Chen, M. Wang, Z. Pan, and A. H. Gandomi, "Multi-population differential evolution-assisted harris hawks optimization: Framework and case studies," *Future Generation Computer Systems*, vol. 111, pp. 175–198, 2020.

[18] B. Tu, F. Wang, Y. Huo, and X. Wang, "A hybrid algorithm of grey wolf optimizer and harris hawks optimization for solving global optimization problems with improved convergence performance," *Scientific Reports*, vol. 13, no. 1, p. 22909, 2023.